PROGRAMMATION EN PYTHON · ING-3 ECE PARIS

SecureLogEngine

Moteur de traitement et d'analyse de flux de logs sécurisé

BENBOUJEMAA Manel
SABITY Brahim
TAGNE CHIBY Ellios

```
PS> python main.py
=== SecureLogEngine : rapport ===
Événements analysés : 49

=== Alertes (4) ===
[brute_force] 203.0.113.45
  6 échecs sur admin
  (attaque réussie)
[blacklist] 192.0.2.150
[blacklist] 198.51.100.77
[privilege] 203.0.113.45
```

Python 3.10+ · Générateurs · Programmation fonctionnelle · Sécurité

CONTEXTE

Les logs : une mine d'informations difficile à exploiter

Volume
Des fichiers de plusieurs gigaoctets : impossible de tout charger en mémoire.

Hétérogénéité
Texte libre, JSON, CSV, et des lignes cassées au milieu.

Menaces cachées
Une force brute ou un scan de ports noyés parmi des milliers de lignes.

Extrait réel de data/samples/auth.log (détails JSON masqués)

```
2026-11-04T08:05:55Z | INFO | 10.0.0.22:50879/TCP | bob | AUTH_SUCCESS | …
2026-11-04T08:12:01Z | WARNING | 203.0.113.45:41001/TCP | admin | AUTH_FAILURE | …
2026-11-04T08:12:03Z | WARNING | 203.0.113.45:41003/TCP | admin | AUTH_FAILURE | …
2026-11-04T08:12:06Z | INFO | 10.0.0.40:52011/TCP | dave | AUTH_SUCCESS | …
2026-11-04T08:12:09Z | WARNING | 203.0.113.45:41009/TCP | admin | AUTH_FAILURE | …
```

En jaune : la même IP échoue plusieurs fois sur le compte admin.

Problématique *Comment analyser des logs volumineux et hétérogènes sans saturer la mémoire, et en extraire des alertes fiables ?*

OBJECTIFS

Six objectifs, une seule commande

Lire en flux

Ligne par ligne avec yield et yield from, sans charger les fichiers.

Structurer

Chaque ligne devient un dictionnaire, la connexion un tuple.

Nettoyer

filter, lambda et map retirent les erreurs et uniformisent.

Analyser

Comptages avec defaultdict, IP avec set et frozenset.

Détecter

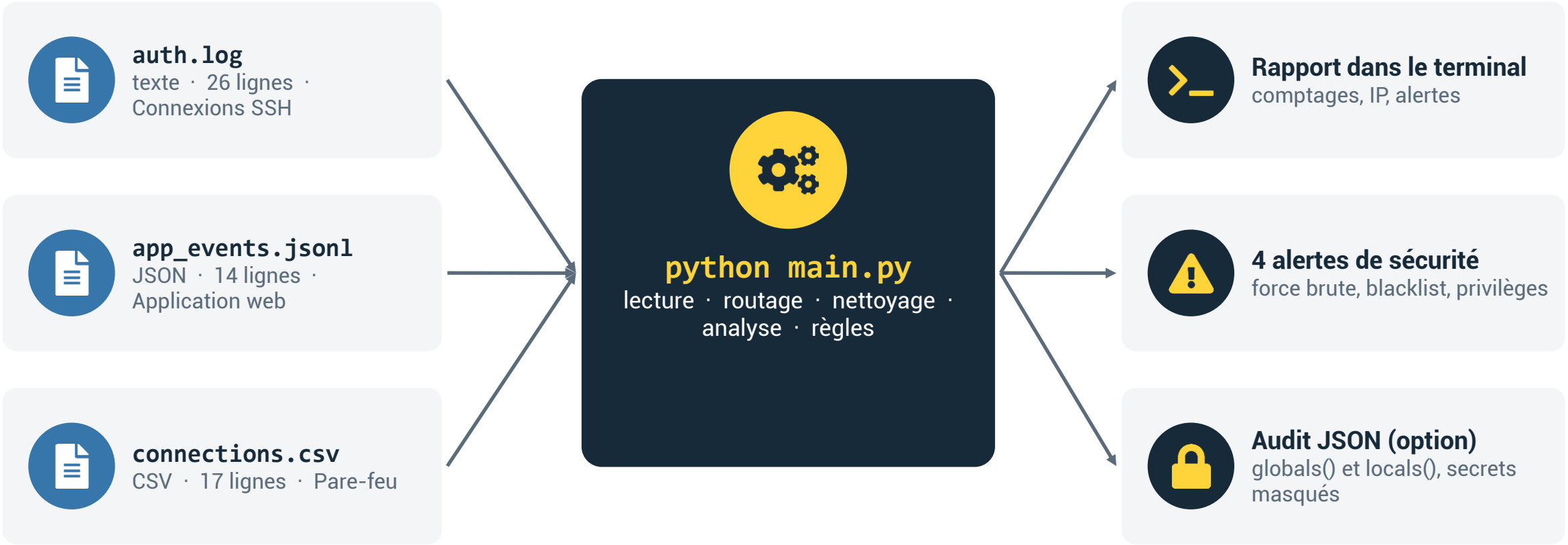
Force brute et moteur de règles configurable (*args, **kwargs).

Auditer et tester

Audit avec globals() et locals(), 132 tests automatisés.

FONCTIONNEMENT GLOBAL

Trois sources en entrée, un rapport en sortie



Options : `[fichiers...]` `--threshold 5` `--blacklist fichier.txt` `--audit audit.json`

SCÉNARIO DE DÉMONSTRATION

Sept démonstrations en direct dans VS Code

1	Lancer l'analyse	`python main.py`	après la slide 5
2	Lecture en streaming	`read_sources + next()`	après la slide 12
3	Filtrage et normalisation	`process_files`	après la slide 12
4	Statistiques et IP	`count_by_user, set`	après la slide 16
5	Force brute	`main.py --threshold 3`	après la slide 16
6	Règles et audit	`main.py --audit`	après la slide 16
7	Tests automatisés	`python -m pytest`	après la slide 18

DÉMONSTRATION

Lancer l'analyse complète

Retour aux slides : slide 7 · Architecture

DÉMO 1 Lancement de l'application

Module `main.py → log_engine/cli.py`

```
PS> python main.py
Événements analysés : 49
Par niveau : INFO 22 | WARNING 22 | ERROR 3 | CRITICAL 2
Par utilisateur : admin 9 | bob 6 | alice 4 | root 3 | ...

=== Adresses IP ===
Blacklistées et actives : 192.0.2.150, 198.51.100.77

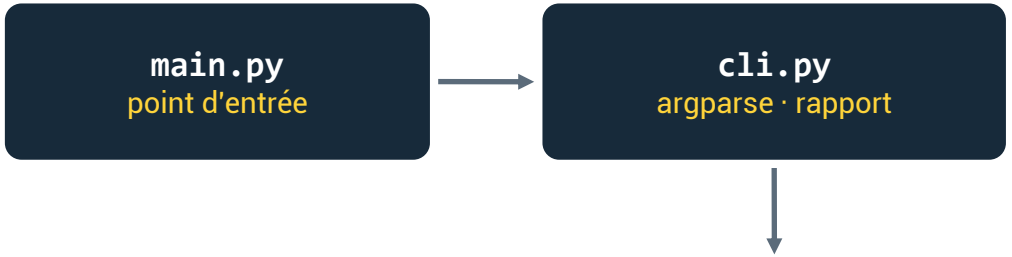
=== Alertes (4) ===
[brute_force] 203.0.113.45 : 6 échecs sur admin (attaque réussie)
[blacklist] 192.0.2.150 : IP blacklistée présente dans les logs
[blacklist] 198.51.100.77 : IP blacklistée présente dans les logs
[privilege] 203.0.113.45 : admin a obtenu des droits élevés
```

À observer

- Une seule commande traite les 3 fichiers : 57 lignes lues, 49 événements valides.
- admin ressort avec 9 événements : c'est le compte attaqué.
- 4 alertes, dont une attaque réussie suivie d'une élévation de privilèges.

ARCHITECTURE GÉNÉRALE

Un package Python, un module par responsabilité



Socle : `models.py` (formats, niveaux) `exceptions.py` (erreurs)

pipeline.py — chaîne de traitement en flux



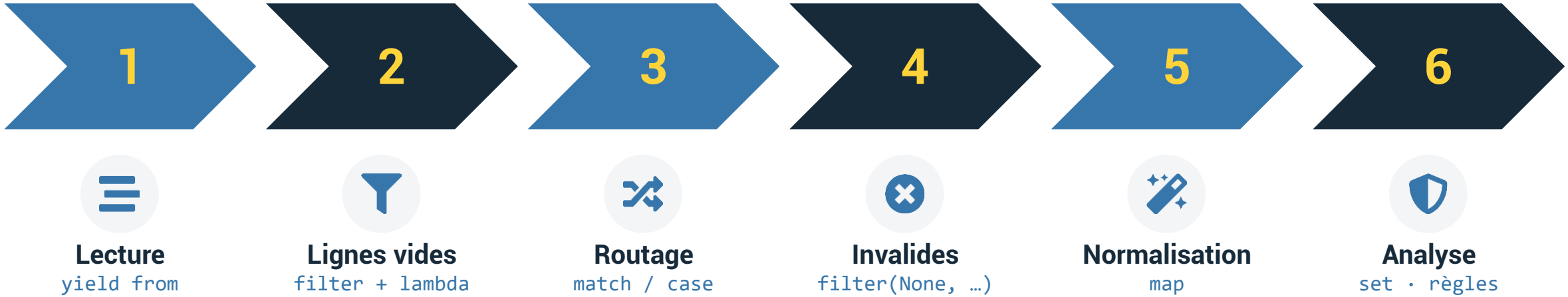
Analyses et sécurité

aggregation.py	ip_analysis.py	bruteforce.py	rules.py	audit.py
defaultdict	set · frozenset	while · break · else	*args · **kwargs	globals() · locals()

14 modules · 387 instructions · 132 tests (pytest) · aucune dépendance externe dans le moteur

PIPELINE DE TRAITEMENT

Le parcours d'une ligne de log



Exemple réel : la ligne 24 de auth.log, volontairement « sale »

```
2026-11-04T08:47:00Z | warning | 10.0.0.22:50990/TCP | Bob  | auth_failure | {"reason": "bad_password"}
```

↓

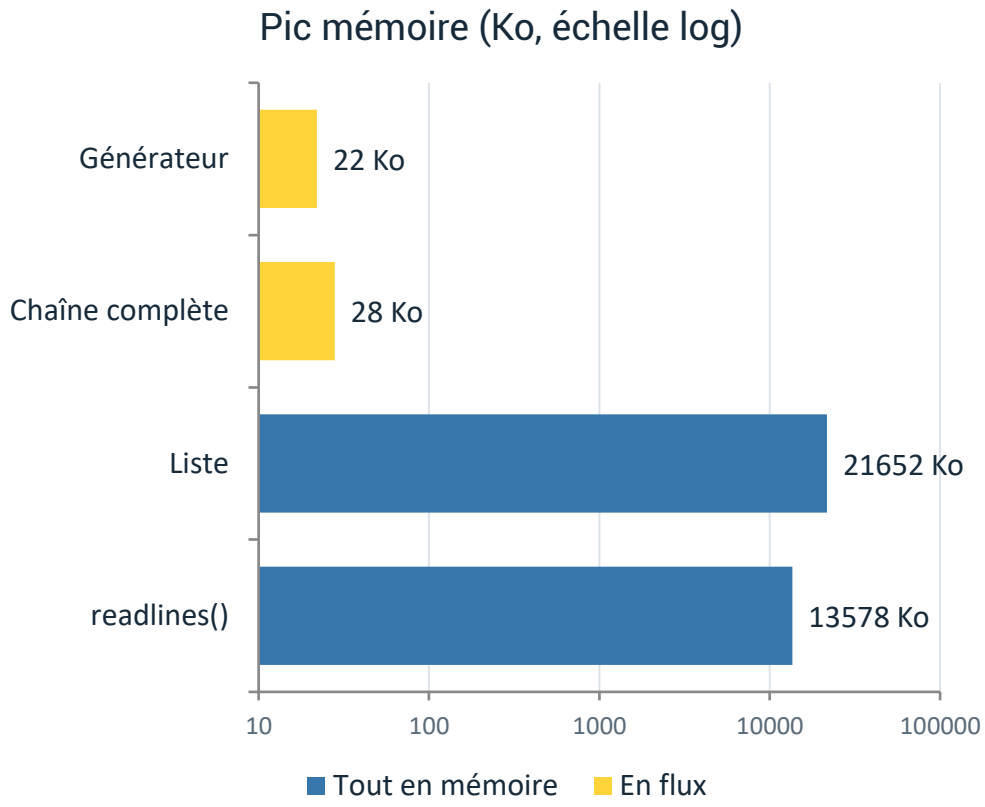
```
{'severity': 'WARNING', 'connection': ('10.0.0.22', 50990, 'TCP'), 'user': 'bob', 'event': 'AUTH_FAILURE', ...}
```

Le flux : une ligne à la fois, quelle que soit la taille

log_engine/pipeline.py `process_files()`

```python
def process_files(paths: list[str]):
    lines = remove_empty_lines(read_sources(paths))
    records = map(lambda item: route_line(item[0], item[2]), lines)
    records = remove_invalid(records)
    records = normalize_records(records)
    return keep_known_severity(records)
```

Chaque étape est paresseuse : read_sources est un générateur, filter et map renvoient des itérateurs. Aucune liste n'est construite : une seule ligne traverse la chaîne à la fois.



Mesures tracemalloc du rapport, fichier de 100 000 lignes (8,8 Mo). « Liste » = list(read_lines(...)). Les valeurs varient selon la machine, l'écart reste d'un facteur 500 à 1 000.

Lire un fichier avec yield, plusieurs avec yield from

ingestion/readers.py `read_lines()`

```python
def read_lines(path: str):
    if not os.path.isfile(path):
        raise SourceNotFoundError(f"Source introuvable : {path}")

    with open(path, encoding="utf-8", errors="replace") as file:
        for number, line in enumerate(file, start=1):
            yield number, line.rstrip("\n")
```

ingestion/readers.py `read_sources()`

```python
def read_sources(paths: list[str]):
    for path in paths:
        try:
            yield from read_source(path)
        except SourceNotFoundError as error:
            logging.warning(f"Source ignorée : {error}")
```

1. **Fichier absent**
   Exception métier SourceNotFoundError, levée au premier next().

2. **with open(...)**
   Le fichier est fermé même si la lecture s'arrête en route.

3. **yield**
   La fonction se met en pause et rend une ligne : une seule en mémoire.

4. **yield from**
   Délègue à un sous-générateur : les fichiers s'enchaînent en un flux.

5. **try / except**
   Un fichier manquant est signalé puis ignoré, les autres sont lus.

match / case : choisir le parseur selon la forme

log_engine/router.py `route_event()`

```
def route_event(data):
    match data:
1       case str():
            return parse_text_line(data)
2       case {
            "timestamp": _,
            "level": _,
            "src_ip": _,
            "src_port": _,
            "protocol": _,
            "event": _,
        }:
            return parse_event_dict(data)
3       case (_, _, _, _, _, _):
            return parse_connection_fields(data)
4       case _:
            return None
```

1. `case str()` chaîne brute (auth.log) → `parse_text_line`
2. `case {...}` dictionnaire avec les 6 clés (JSON) → `parse_event_dict`
3. `case (_, _, _, _, _, _)` tuple de 6 champs (CSV) → `parse_connection_fields`
4. `case _` toute autre forme → `None, ligne rejetée`

log_engine/parsers.py `make_connection() — fin`

```
    return (ip, port, protocol)
```

Résultat : un dictionnaire, avec un tuple immuable (IP, port, protocole) pour la connexion.

filter retire, map transforme, lambda précise

log_engine/filters.py 3 filtres

```
1  def remove_empty_lines(lines):
       return filter(lambda item: item[2].strip() != "", lines)

2  def remove_invalid(records):
       return filter(None, records)

3  def keep_known_severity(records):
       return filter(
           lambda record: record["severity"] in SEVERITY_LEVELS, records
       )
```

log_engine/normalizer.py normalize_records()

```
4  def normalize_records(records):
       return map(normalize_record, records)
```

1. **filter + lambda**
   garde les lignes non vides
2. **filter(None, …)**
   retire les None du parseur
3. **lambda record: …**
   garde les niveaux connus
4. **map(fonction, …)**
   uniformise : WARNING, bob, f-string

```
warning · Bob · auth_failure
→  [WARNING] 10.0.0.22 bob AUTH_FAILURE
```

DÉMONSTRATION

Le flux et la normalisation en action

Retour aux slides : slide 14 · Statistiques

DÉMO 2 Lecture en streaming

Module `log_engine/ingestion/readers.py`

```
PS> python -c "from log_engine.ingestion.readers
    import read_sources; lines = read_sources([
    'data/samples/auth.log']); print(lines);
    print(next(lines)); print(next(lines))"
<generator object read_sources at 0x...>
('auth.log', 1, '2026-11-04T08:00:00Z | INFO | ...')
('auth.log', 2, '2026-11-04T08:03:12Z | INFO | ...')
```

À observer

- L'appel renvoie un générateur : rien n'est lu.
- Chaque next() lit une seule ligne, sous forme de tuple.
- Option : python -m scripts.memory_demo.

DÉMO 3 Filtrage et transformation

Module `log_engine/pipeline.py`

```
PS> Select-String data\samples\auth.log
    -Pattern "Bob","BOB" -CaseSensitive
...| warning | 10.0.0.22:50990/TCP | Bob  | auth_failure
...| Info | 10.0.0.22:50992/TCP |  BOB | Auth_Success
PS> python -c "from log_engine.pipeline import ..."
[WARNING] 10.0.0.22 bob AUTH_FAILURE
[INFO] 10.0.0.22 bob AUTH_SUCCESS
[WARNING] 10.0.0.22 bob AUTH_FAILURE
[INFO] 10.0.0.22 bob AUTH_SUCCESS
```

À observer

- Les 2 lignes brutes de Bob sont mal écrites.
- Après la chaîne, ses 4 événements (2 propres, 2 sales) ont le même format.
- Commande complète dans le guide.

Compter avec defaultdict, comparer avec des ensembles

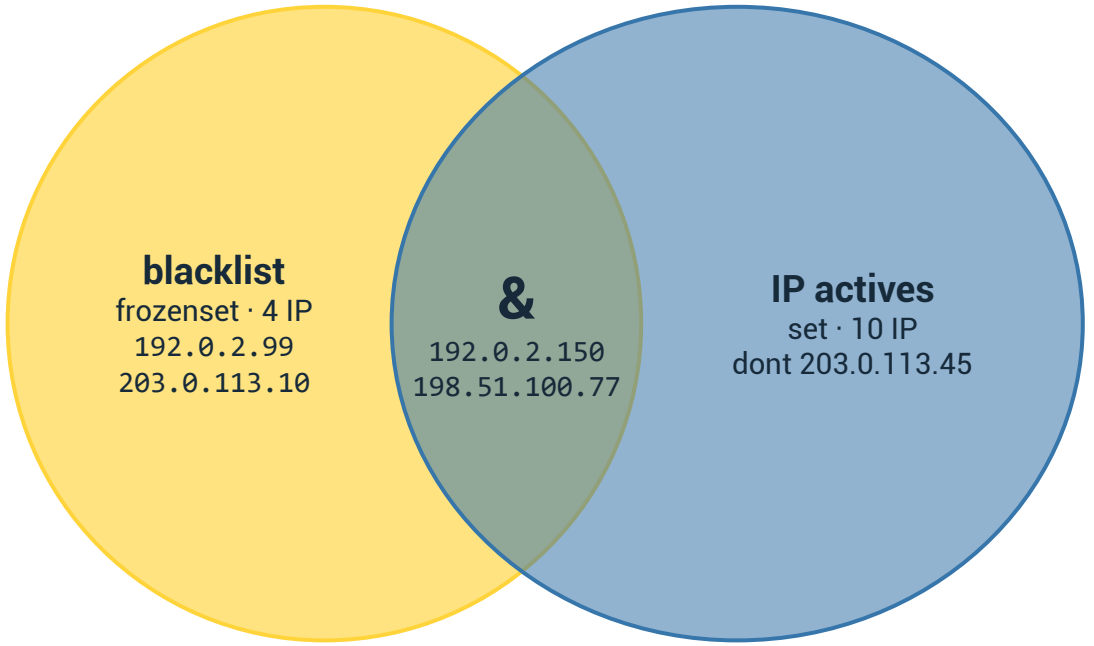
log_engine/aggregation.py `count_by_field()`

```python
def count_by_field(records, field: str):
    counts = defaultdict(int)
    for record in records:
        value = record[field]
        if value is None:
            continue
        counts[value] += 1
    return counts
```

log_engine/ip_analysis.py `compare_with_blacklist()`

```python
def compare_with_blacklist(blacklist, active, errors):
    return {
        "blacklisted_and_active": blacklist & active,
        "suspects_not_blacklisted": errors - blacklist,
        "new_blacklist": blacklist | errors,
        "in_only_one": blacklist ^ active,
    }
```

1 aucune KeyError **2** on saute les None **3** compteur incrémenté



`blacklist & actives`	IP bannies vues dans les logs : 2
`erreurs - blacklist`	IP suspectes non bannies : 6
`blacklist | erreurs`	blacklist proposée : 10 IP
`blacklist ^ actives`	présentes d'un seul côté : 10 IP

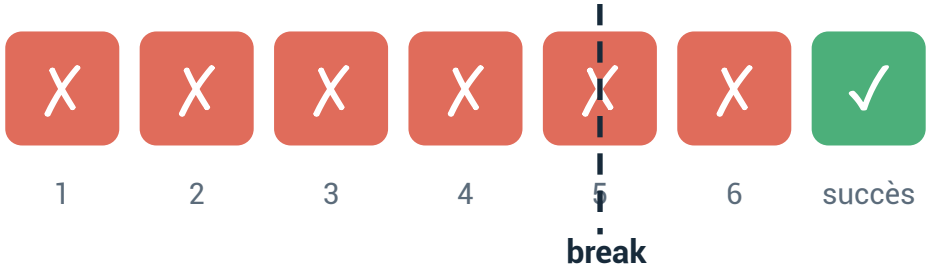
Détecter une force brute : while, break et else

log_engine/bruteforce.py `has_consecutive_failures()`

```python
def has_consecutive_failures(attempts: list, threshold: int):
    streak = 0
    index = 0
    while index < len(attempts):
        if attempts[index]:
            streak += 1
        else:
            streak = 0
        if streak >= threshold:
            break
        index += 1
    else:
        return False
    return True
```

1 2 3

Les tentatives de 203.0.113.45 dans auth.log (seuil 5)



Le succès de dave, depuis une autre IP au milieu de l'attaque, n'interrompt pas la série : les tentatives sont regroupées par IP.

1. **while**
   parcourt les tentatives d'une IP
2. **break**
   seuil atteint : attaque prouvée, on arrête
3. **else de boucle**
   cycle complet sans break : rien trouvé

```
describe_attacks() →
'failures': 6, 'succeeded': True
```

Des règles branchées dynamiquement, un audit sans secrets

log_engine/rules.py `apply_security_rules()`

```
1  def apply_security_rules(records, *args, **kwargs):
       records = list(records)
       alerts = []
       for rule in args:
           try:
2              alerts.extend(rule(records, **kwargs))
3          except Exception as error:
               logging.warning(f"Règle {rule.__name__} ignorée : {error}")
       return alerts
```

log_engine/cli.py `main() — audit`

```
export_audit_report(args.audit, globals(), locals())
```

Dans audit.json : records → 49 éléments, blacklist → frozenset de 4 éléments. Une variable nommée api_token ou password est masquée : ***.

log_engine/cli.py `main() — appel`

```
alerts = apply_security_rules(
    records,
    rule_brute_force,
    rule_blacklisted_ips,
    rule_privilege_escalation,
    threshold=args.threshold,
    blacklist=blacklist,
)
```

1. ***args**
   les règles, en nombre variable : un tuple de fonctions
2. ****kwargs**
   la configuration : threshold, blacklist…
3. **try / except**
   une règle défectueuse n'arrête pas les autres

DÉMONSTRATION

Statistiques, force brute, règles

Retour aux slides : slide 18 · Résultats

DÉMO 4 Analyse statistique

Module `aggregation.py · ip_analysis.py`

```
PS> python -c "...count_by_user..."
{'alice': 4, 'bob': 6, 'admin': 9,
 'dave': 2, 'root': 3, 'carol': 3}
['192.0.2.150', '198.51.100.77']
['192.0.2.99', '203.0.113.10']
```

À observer

- defaultdict : un compteur par utilisateur.
- blacklist & actives : 2 IP.
- blacklist - actives : 2 IP jamais vues.

DÉMO 5 Force brute

Module `bruteforce.py`

```
PS> python main.py --threshold 3
=== Alertes (5) ===
[brute_force] 198.51.100.23 :
  3 échecs sur root (attaque échouée)
[brute_force] 203.0.113.45 :
  6 échecs sur admin (attaque réussie)
```

À observer

- Seuil 5 : 1 attaque. Seuil 3 : 2.
- La nouvelle IP a visé root sans succès.

DÉMO 6 Règles et audit

Module `rules.py · audit.py`

```
PS> python main.py --audit audit.json
Audit exporté dans audit.json
PS> Get-Content audit.json
"records": {"type": "list",
            "value": "49 éléments"}
PS> Remove-Item audit.json
```

À observer

- 4 alertes de 3 règles différentes.
- globals et locals de main() exportés.
- Supprimer le fichier ensuite.

RÉSULTATS VÉRIFIÉS

Ce que le moteur détecte réellement

57 → 49

lignes lues → événements valides
8 lignes cassées rejetées

4

alertes de sécurité
3 règles différentes

132

tests automatisés
tous verts

100 %

de couverture
du package log_engine

```
PS> python main.py
[brute_force] 203.0.113.45 : 6 échecs sur admin (attaque réussie)
[blacklist] 192.0.2.150 : IP blacklistée présente dans les logs
[blacklist] 198.51.100.77 : IP blacklistée présente dans les logs
[privilege] 203.0.113.45 : admin a obtenu des droits élevés
```



Scénario d'attaque reconstitué
203.0.113.45 échoue 6 fois sur admin, réussit, puis lance sudo. Le scanner 198.51.100.77, déjà blacklisté, est repéré. 198.51.100.23 (3 échecs) reste sous le seuil.

DÉMONSTRATION

Les tests automatisés

Retour aux slides : slide 20 · Limites

DÉMO 7 Exécution des tests

Module `tests/ (15 fichiers)`

```
PS> python -m pytest
collected 132 items
tests/test_aggregation.py ......              [  4%]
tests/test_bruteforce.py ..............       [ 21%]
...
tests/test_pipeline.py .......                [ 70%]
...
========== 132 passed in 3.81s ==========

PS> python -m pytest --cov=log_engine
TOTAL         387      0   100%
```

À observer

- Tests unitaires (une fonction) et d'intégration (toute la chaîne, la CLI).
- Un test mesure la mémoire avec tracemalloc.
- Toujours python -m pytest : Windows peut bloquer pytest.exe.

REGARD CRITIQUE

Limites, difficultés et pistes d'amélioration

Limites

- Fichiers lus l'un après l'autre, sans fusion par date.
- Les analyses travaillent sur la liste des événements filtrés.
- Secrets de l'audit repérés par le nom des variables.
- Force brute sans fenêtre de temps.

Difficultés rencontrées

- Windows bloque pytest.exe et une DLL de matplotlib.
- Mesure mémoire différente selon la plateforme.
- Bug trouvé par nos tests : JSON [1, 2, 3, 4, 5, 6].
- Migration du dépôt Git en cours de projet.

Pistes d'amélioration

- Fusion chronologique des flux.
- Fenêtre de temps : X échecs en Y secondes.
- Export du rapport d'analyse en JSON.
- Lecture en continu d'un fichier qui grossit.

BILAN TECHNIQUE

Toutes les notions du cahier des charges, au service du moteur

✓	**Générateurs** readers.py	`yield, yield from`	
✓	**Fonctionnel** filters · normalizer	`filter, map, lambda`	
✓	**Pattern matching** router.py	`match / case`	
✓	**Structures** parsers · ip_analysis	`dict, tuple, set, frozenset`	
✓	**Accumulation** aggregation.py	`defaultdict`	
✓	**Contrôle de flux** bruteforce.py	`while, break, continue, else`	
✓	**Fonctions dynamiques** rules.py	`*args, **kwargs`	
✓	**Introspection** audit.py	`globals(), locals()`	

Génie logiciel : environnement virtuel · requirements.txt · Git et branches · 132 tests pytest · PEP 8 avec ruff · gestion des exceptions

CONCLUSION

Un moteur complet, en flux, testé

1 000×
moins de mémoire qu'un chargement complet

4
alertes qui reconstituent l'attaque

132
tests, 100 % de couverture

Ouverture : le même principe de flux s'applique à l'analyse de logs en continu, comme le font les outils de supervision (SIEM).

Merci de votre attention · Questions ?

github.com/brahimsbt/SecureLogEngine